Latviz: A Complete Process Visualization for Post-Quantum Standards ML-KEM and ML-DSA Using Interactive Interface

Jeslyn Theodora and Amril Syalim

Faculty of Computer Science, University of Indonesia.

Abstract. Post-quantum cryptographic algorithms are gaining relevance as prevalent security measures are found vulnerable to quantum computers. Among them are the key encapsulation standard ML-KEM and the digital signature standard ML-DSA released by NIST (National Institute of Standards and Technology) in 2024. However despite it's relevance, there are few resources dedicated to fully describing the algorithms behind them that are easily followed. This paper details the creation of'Latviz', a digital visualization tool meant to help learners understand the entire process behind the ML-KEM and ML-DSA standards. It takes into account the qualities that have a greater impact on the process of learning algorithms by employing interactivity and visual abstractions.

Keywords: Cryptography · Post-Quantum · Algorithm Visualization · Visualization · ML-KEM · ML-DSA · Lattice-based Cryptography

1 Introduction

Ever since quantum computing was first theorized in the early 1980s [1, 2], its potential capabilities have been continuously theorized and expanded upon. From improving computer simulation using quantum-based phenomena [3] to applying quantum theory to improve information security [4, 5], quantum computing has had profound implications on entire fields. Among them are its implications on cryptography. In 1994, Peter Shor proposed a quantum algorithm that, given a sufficiently powerful quantum computer, might be used to break protocols like RSA [6]. Shor's algorithm and others like it significantly lower the difficulty of problems such as the integer factorization problem and discrete logarithm problem, which the most widely used public key algorithms base their difficulty on. This means that common security protocols are at risk of being broken sometime in the near future. Because of this, cryptology experts around the world have looked into new protocols that might have the potential to stay secure despite the continued development into quantum computers.

NIST (National Institute of Standards and Technology) is one of the organizations at the forefront of the effort. In 2016, they put out a call for proposals that might be used in post-quantum cryptography. Eight years later, they released three standards based on those submissions, two of which are ML-KEM

and ML-DSA, based on the Kyber and Dilithium algorithms respectively [7, 8]. ML-KEM is used for key encapsulation, while ML-DSA is used for digital signatures. Both fall under the umbrella of lattice-based cryptography

However, despite their importance, there are a limited number of resources available for learning about them. Outside of NIST's official releases and lectures, most of they come in the from of summaries of said releases and lectures, public implementations, and simplified diagrams or visualizations. The visualization and exploration tool pqc-vizz for example, provides an interactive working implementation for ML-KEM and ML-DSA that describes the overall process of using them, but skips over most of their inner workings such as their background calculations [9]. Daeukun Kang's ML-KEM from Scratch details the entire process of ML-KEM as well as the context and mathematical grounding behind it, including a working Python implementation [10]. However, it assumes that the user has a working understanding of coding; otherwise, it is nothing more than a particularly structured summary. Other resources are similarly lacking in that they do not offer anything beyond a summary of NIST's release or they simplify their explanations to only the outermost layers.

Thus, this work aims to develop a visualization tool Latviz that allows users to view the complete process behind ML-KEM and ML-DSA in a way that is still digestible to most people. This is done by displaying the inner mechanisms of both algorithms such that users unfamiliar with cryptography can still follow the explanations, and as well as going through both processes in their entirety instead of focusing only on the inputs and outputs. To achieve this, the tool implements a set of features designed to improve accesibility while preserving technical accuracy.

- **Complete algorithmic breackdown**: Latviz provides a more in-depth breakdown of the ML-KEM and ML-DSA standards compared to existing resources. The variables and algorithms that are involved in the each stage of both standards are elaborated upon in a step-by-step manner, with the explanations simplified as to be digestible without complete understanding of the mathematical grounding required to otherwise understand it.
- **Interactive visual interface**: Merely viewing a visualization does not give significant advantages over conventional methods such as following lectures or reading in how they impact learning algorithms. The most successful uses of visualization tools for learning algorithms are when the tools are used as a vehicle for actively engaging in the process of learning said algorithms [11], such as in the form of analyses of algorithmic behavior through manipulating functions and variables in the Desmos Graphic Calculator where users can directly see how certain functions and variables affect graphs. While ML-KEM and ML-DSA doesn't allow for as much interactivity as most of the variables and functions are either set or completely randomized, Latviz provides interactivity in the form of the manipulation of parameter sets, stepping through operations, and the inspection of internal states.

To evaluate the effectiveness of the tool,

2 Related Works

The following section discusses inner workings of ML-KEM and ML-DSA in more depth, as well as works surrounding the effective visualization of algorithms.

2.1 Lattice-Based Cryptography

A lattice is a discrete subset of n-dimensional vectors of real numbers $\mathbb{R}^n$ formed by all integer linear combinations of a set of linearly independent basis vectors. Formally, given a basis $\mathbf{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_k\} \subset \mathbb{R}^n$, a lattice $\mathcal{L}$ is defined as:

$$\mathcal{L} = \left\{ \sum_{i=1}^{k} z_i \mathbf{b}_i \;\middle|\; z_i \in \mathbb{Z} \right\}$$

Lattices can be visualized as regularly spaced grids extending in multiple dimensions, but their structure becomes increasingly complex and difficult to visualize in high-dimensional spaces, and especially beyond the third dimension.

Lattice-based cryptography relies on the presumed hardness of certain computational problems defined over lattices. Two of the most important problems are the Shortest Vector Problem (SVP), which asks for the shortest non-zero vector in a lattice, and the Learning With Errors (LWE) problem, which involves solving systems of linear equations that have small random noise. As of now, no algorithms exist that can solve these problems efficiently even with quantum computers, and they are tentatively believed to be quantum resistant.

Practical lattice-based schemes often use structured variants such as Ring-LWE and Module-LWE (MLWE), which operate over polynomial rings. These variants significantly improve efficiency while retaining the core hardness properties. MLWE in particular provides a balance between performance and conservative security assumptions and forms the basis of several standardized post-quantum schemes, including ML-KEM and ML-DSA.

It should be noted that both ML-KEM and ML-DSA employ Number Theoretic Transform (NTT), a generalization of Discrete Fourier Transform (DFT) to finite fields, to accelerate polynomial multiplication. The NTT representation of a polynomial will be in hat notation(e.g., $\hat{A}$).

2.2 ML-KEM - FIPS 203

ML-KEM, also referred to as Module-Lattice-Based Key-Encapsulation Mechanism Standard, is a standard set by NIST's FIPS 203 publication that specifies a set of algorithms meant to be used to establish a shared private key between two parties. This standard was proposed in response to the potential unreliability of modern cryptographic key systems such as RSA in the face of quantum computers.

ML-KEM is a variant of the CRYSTALS-Kyber algorithm [12], which is itself based on the hardness of solving MLWE, which is based on the presumed hardness of certain computational problems in module lattices. It can be described

as a generalization of the Learning with Errors (LWE) problem as previously described. Both LWE and MLWE are widely considered to be quantum resistant, in that there are no known quantum algorithms that can efficiently solve it.

ML-KEM and other KEMs (Key Encapsulation Mechanisms) typically consist of three main algorithms: a key generation, key encapsulation, and key decapsulation algorithm.

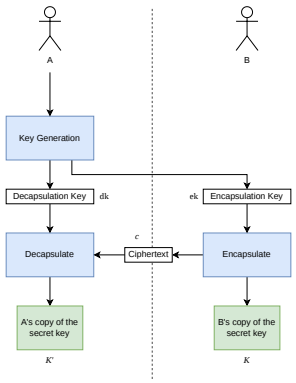


Fig. 1: Simplified key establishment process using KEM

As shown in figure 1, KEMs are used to establish a shared private key between two parties. Party A begins by running a key generation algorithm that produces an encapsulation (public) key and decapsulation (private) key. When party B accepts the encapsulation key, they can run the encapsulation algorithm with it as an input, which produces a copy of the two parties' shared secret key K and its associated ciphertext c. Party B sends the ciphertext to party A, and party A runs the decapsulation algorithm using the decapsulation key and the ciphertext, producing their own copy K' of the shared secret key.

They also have multiple parameter sets, of which ML-KEM has three. Each set specifies the underlying ring dimension, module rank, noise distributions, and compression parameters. Increasing these parameters results in larger matrices, vectors, and noise levels, which enhances security at the cost of computational efficiency. In order of least to most secure and most to least efficient, they are ML-KEM-512, ML-KEM-768, and ML-KEM-1024.

Table 1: Parameter sets for ML-KEM (Kyber) security levels

	n	q	k	η_1	η_2	d_u	d_v	*Required RBG strength (bits)*
ML-KEM-512	256	3329	2	3	2	10	4	128
ML-KEM-768	256	3329	3	2	2	10	4	192
ML-KEM-1024	256	3329	4	2	2	11	5	256

Each parameter set is comprised of the five individual parameters k, η_1, η_2, d_u, and d_v, and two constants n and q.

- k determines the dimensions of the matrix A that appears in K-PKE.KeyGen and K-PKE.Encrypt, as well as the dimensions of vectors s and e in K-PKE.KeyGen and vectors y and e_1 in K-PKE.Encrypt. An increase in k comes with an increase in security level, key sizes, and ciphertext sizes, but also computational cost.
- η_1 specifies the distribution for generating vectors s and e in K-PKE.KeyGen and the vector y in K-PKE.Encrypt. Essentially, increasing η_1 produces larger noise terms and increases security level at the cost of efficiency.
- η_2 specifies the distribution for generating vectors error e_1 and e_2 in K-PKE.Encrypt.
- d_u and d_v are used as inputs and parameters for utility functions Compress, Decompress, ByteDecode, and ByteEncode in K-PKE.KeyGen and K-PKE.Encrypt.
- Constants n and q are the same between the three approved parameter sets. n denotes the degree of the polynomial ring for the scheme, while q is the modulus used for modular arithmetic.

2.2.1 ML-KEM Key Generation

The key generation algorithm ML-KEM.Keygen in ML-KEM produces an encapsulation key and decapsulation key using internally generated 32 byte seeds d and z.

Algorithm 1: ML-KEM.KeyGen()

Output: encapsulation key ek $\in \mathbb{B}^{384k+32}$
Output: decapsulation key dk $\in \mathbb{B}^{768k+96}$

1 $d \leftarrow^{\$} \mathbb{B}^{32}$ /* d is 32 random bytes */
2 $z \leftarrow^{\$} \mathbb{B}^{32}$ /* z is 32 random bytes */
3 **if** $d ==$ NULL *or* $z ==$ NULL **then**
4 | **return** $\perp$ /* return an error indication if random bit generation failed */
5 (ek, dk) $\leftarrow$ ML-KEM.KeyGen_internal(d, z)
6 **return** (ek, dk)

Algorithm 2: ML-KEM.KeyGen_internal(d, z)

Input : randomness $d \in \mathbb{B}^{32}$
Input : randomness $z \in \mathbb{B}^{32}$
Output: encapsulation key $\text{ek} \in \mathbb{B}^{384k+32}$
Output: decapsulation key $\text{dk} \in \mathbb{B}^{768k+96}$

1 $(\text{ek}_{\text{PKE}}, \text{dk}_{\text{PKE}}) \leftarrow$ K-PKE.KeyGen(d) `/* run key generation for K-PKE */`
2 $\text{ek} \leftarrow \text{ek}_{\text{PKE}}$ `/* KEM encaps key is just the PKE encryption key */`
3 $\text{dk} \leftarrow (\text{dk}_{\text{PKE}} \| \text{ek} \| H(\text{ek}) \| z)$ `/* KEM decaps key includes PKE decryption key */`
4 **return** (ek, dk)

Assuming the generated random seeds d and z are valid, the seed d is used to run Kyber's key generation algorithm K-PKE.KeyGen, which outputs the encryption key and decryption key. The encapsulation key is the outputted encryption key of K-PKE. The decapsulation key consists of the outputted decryption key of K-PKE, the encapsulation key, a hash of the encapsulation key, and the random 32-byte value seed z.

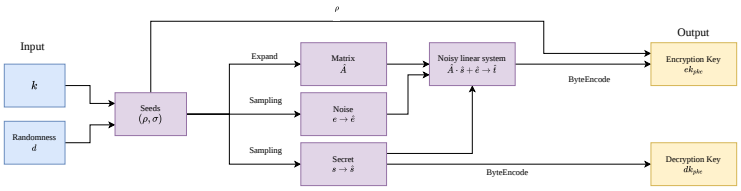


Fig. 2: Kyber Key Generation Algorithm K-PKE.KeyGen

K-PKE.KeyGen begins by generating 32 byte seeds ρ and σ from hashing the input d and k from the chosen parameter set. From the seeds, matrix A is determinalistically expanded from ρ and secret vectors s and noise e are sampled from centered binomial distributions using randomness expanded from σ. The three together form a noisy linear combination that computes the part t of the encryption key. The seed for matrix A is then appended to t to create the encryption key, while vector s makes up the decryption key. Both are encoded into the appropriate byte arrays, then output.

2.2.2 ML-KEM Encapsulation

The key encapsulation algorithm ML-KEM.Encaps accepts an encapsulation key and random value m as input and produces a shared key and its associated ciphertext.

Algorithm 3: ML-KEM.Encaps(ek)

Checked input: encapsulation key ek $\in \mathbb{B}^{384k+32}$
Output : shared secret key $K \in \mathbb{B}^{32}$
Output : ciphertext $c \in \mathbb{B}^{32(d_u k+d_v)}$

$m \leftarrow^{\$} \mathbb{B}^{32}$ `/* m is 32 random bytes */`
if $m ==$ NULL **then**
  return $\perp$
$(K, c) \leftarrow$ ML-KEM.Encaps_internal(ek, m) **return** (K, c)

Algorithm 4: ML-KEM.Encaps_internal(ek, m)

Input : encapsulation key ek $\in \mathbb{B}^{384k+32}$
Input : randomness $m \in \mathbb{B}^{32}$
Output: shared secret key $K \in \mathbb{B}^{32}$
Output: ciphertext $c \in \mathbb{B}^{32(d_u k+d_v)}$

$(K, r) \leftarrow G(m \,\|\, H(\text{ek}))$ `/* derive shared secret key K and randomness r */`
$c \leftarrow$ K-PKE.Encrypt(ek, m, r) `/* encrypt m using K-PKE with randomness r */`
return (K, c)

A copy of shared secret K and the randomness r used during encryption are derived from the encapsulation key and m via hashing. The randomness r, the encapsulation key, and m are then used as input for Kyber's encryption algorithm, K-PKE-Encrypt, which will encrypt m into ciphertext.

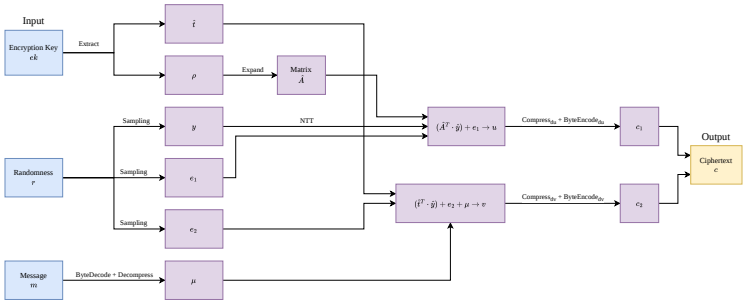


Fig. 3: Kyber Encryption Algorithm K-PKE.Encrypt

From the encryption key, $\hat{t}$ and seed ρ are extracted. Matrix $\hat{A}$ is then re-expanded from ρ. If extracted correctly, both $\hat{t}$ and $\hat{A}$ should be the same as they were in K-PKE.KeyGen.

Following that, the vector y and noise terms e_1 and e_2 are sampled from centered binomial distribution using pseudorandomness from input randomness

r. The ciphertext is then computed in two parts through the noisy equations $(\hat{A}^T \cdot \hat{y}) + e_1$ and $(\hat{t}^T \cdot \hat{y}) + e_2$. The encoding μ of message m is added to the latter formula. The resulting pair (u, v) is compressed and converted into a byte array before being output as the ciphertext.

2.2.3 ML-KEM Decapsulation

The decapsulation algorithm ML-KEM.Decaps accepts a decapsulation key and ciphertext generated by ML-KEM as input and outputs a shared secret key.

Algorithm 5: ML-KEM.Decaps(dk, c)

Checked input: decapsulation key $\text{dk} \in \mathbb{B}^{768k+96}$
Checked input: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$
Output : shared secret key $K \in \mathbb{B}^{32}$

$K' \leftarrow$ ML-KEM.Decaps_internal(dk, c) **return** K'

Algorithm 6: ML-KEM.Decaps_internal(dk, c)

Input : decapsulation key $\text{dk} \in \mathbb{B}^{768k+96}$
Input : ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$
Output: shared secret key $K \in \mathbb{B}^{32}$

$\text{dk}_{\text{PKE}} \leftarrow \text{dk}[0 : 384k]$ /* extract PKE decryption key */
$\text{ek}_{\text{PKE}} \leftarrow \text{dk}[384k : 768k + 32]$ /* extract PKE encryption key */
$h \leftarrow \text{dk}[768k + 32 : 768k + 64]$ /* extract hash of PKE encryption key */
$z \leftarrow \text{dk}[768k + 64 : 768k + 96]$ /* extract implicit rejection value */
$m' \leftarrow$ K-PKE.Decrypt($\text{dk}_{\text{PKE}}, c$) /* decrypt ciphertext */
$(K', r') \leftarrow G(m' \parallel h)$ $\overline{K} \leftarrow J(z \parallel c)$ $c' \leftarrow$ K-PKE.Encrypt($\text{ek}_{\text{PKE}}, m', r'$) /* re-encrypt using derived randomness r' */
if $c \neq c'$ **then**
    $K' \leftarrow \overline{K}$ /* if ciphertexts do not match, "implicitly reject" */
return K'

From the decapsulation key, the PKE encryption key ek_{PKE}, PKE decryption key dk_{PKE}, the hash value h of the PKE encryption key, and random value z are extracted. The ciphertext is decrypted through the K-PKE.Decrypt algorithm using the extracted decryption key, producing the plaintext message m'. The algorithm then re-encrypts the produced message is into a new output ciphertext c' along with computing candidate shared secret key K' as done in the encapsulation stage. If the newly produced ciphertext does not match the input ciphertext, the algorithm performs an 'implicit rejection', in which the value of K' is replaced with a hash of c appended to z. Otherwise, the decapsulation returns shared secret K' unchaged.

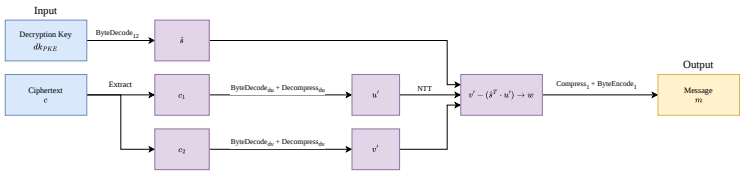


Fig. 4: Kyber Decryption Algorithm K-PKE.Decrypt

The K-PKE.Decrypt algorithm takes a decryption key and the corresponding ciphertext as inputs and extracts the secret variable s and parts c_1 and c_2 from them respectively. Part c_1 is converted into u' while c_2 is converted into v'. The true constant term v is computed through $s^t \cdot u'$. The resulting v is then subtracted from v', outputting w, which is decoded into a plaintext message m.

2.3 ML-DSA - FIPS 204

ML-DSA, also known as Module-Lattice-Based Digital Signature Standard, is a standard set by NIST's FIPS 204 publication that defines a method for digital signature generation and methods for the verification and validation of those signatures. The standard was proposed in response to the potential unreliability of modern digital signatures in the face of steady development of quantum computers.

ML-DSA is derived from the CRYSTALS-Dilithium algorithm [13], whose security is based on the computational hardness of the MLWE and a nonstandard variant of MSIS (Module-Short Integer Solution) called SelfTargetMSIS. Both MLWE and MSIS (which relies on the hardness of SVP) are considered quantum resistant, and ML-DSA inherets that quality.

ML-DSA and other digital signature schemes typically consist of three algorithms: a key generation algorithm, signing algorithm, and signature verifying algorithm.

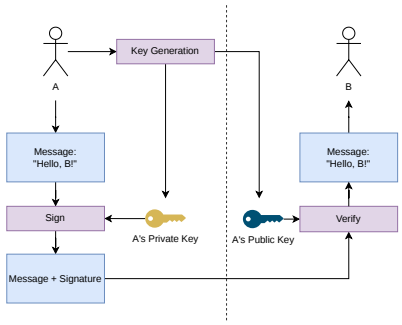


Fig. 5: Simplified digital signature scheme

As shown in figure 5, digital signature schemes are used to provide assurance that a digital message or document is authentic. Party A begins by running a key generation algorithm that produces a public and private key. The private key is kept by party A, while the public key is given to party B. When party A sends a message or document to party B, they run the signing algorithm using their private key and the message, which produces a signature. Party B then can, upon receiving it, verify the message's authenicity using the corresponding public key.

ML-DSA also has three approved parameter sets. In order of least to most secure and most to least efficient, they are ML-DSA-44, ML-DSA-65, and ML-DSA-87.

Table 2: Parameter values for the ML-DSA parameter sets

	q	ζ	d	τ	λ	γ_1	γ_2	(k,ℓ)	η	β	ω
ML-DSA-44	8380417	1753	13	39	128	2^{17}	$\frac{q-1}{88}$	$(4,4)$	2	78	80
ML-DSA-65	8380417	1753	13	49	192	2^{19}	$\frac{q-1}{32}$	$(6,5)$	4	196	55
ML-DSA-87	8380417	1753	13	60	256	2^{19}	$\frac{q-1}{32}$	$(8,7)$	2	120	75

Each parameter set is comprised of the variables τ, λ, γ_1, γ_2, (k,ℓ), η, β, and ω, along with the constants q, ζ, and d.

- τ determines the number of ± 1's in polynomial c in the signing algorithm.
- λ determines the collision strength of $\tilde{c}$ by controlling the output length of the hash function that produces it.
- γ_1 determines the coefficent range of polynomial vector y in the signing algorithm.
- γ_2 determines the low-order rounding range used in the decomposition and hint-generation procedures during signing and verification.

- (k, ℓ) determine the dimensions of matrix A and the sizes of vectors used throughout the scheme. Larger values increase security at the cost of larger signatures and higher computational costs.
- η determines the coefficient range of the secret vectors s_1 and s_2. The coefficients are sampled from a small bounded distribution to ensure the resulting lattice vectors remain short.
- β defines the rejection bound used during signing. It limits the allowable size of coefficients in intermediate values to prevent leakage of secret information.
- ω determines the maximum number of nonzero coefficients allowed in the hint vector h. This ensures that the hint remains sparse and compact.
- q is the modulus used for all polynomial coefficient arithmetic.
- ζ is a primitive 512th root of unity in $\mathbb{Z}_q$ used in the Number Theoretic Transform (NTT).
- d determines the number of low-order bits discarded during the `Power2Round` decomposition procedure used in public key compression.

2.3.1 ML-DSA Key Generation

The key generation algorithm ML-DSA.KeyGen chooses a random 32 byte seed ξ and uses it to generate a public key and private key. The seed itself is generated using an approved RBG (Random Bit Generator) such as described by Barker et al [14–16].

Algorithm 7: ML-DSA.KeyGen()

Output: public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$
Output: private key $sk \in \mathbb{B}^{32+32+64+32\cdot((\ell+k)\cdot\text{bitlen}(2\eta)+dk)}$

$\xi \leftarrow \mathbb{B}^{32}$ `/* choose random seed */`
if $\xi =$ NULL **then**
  return $\perp$ `/* return an error indication if random bit generation failed */`
return ML-DSA.KeyGen_internal(ξ)

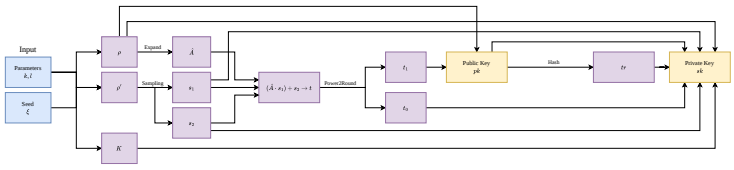


Fig. 6: ML-DSA Internal Keygen Algorithm ML-DSA.KeyGen_internal

The seed ξ is expanded into 32 byte random seed ρ, 64 byte random seed ρ', and 32 byte random seed K. Seed ρ is expanded into matrix A through

sampling, and secret polynomial vectors s_1 and s_2 are sampled using seed ρ'. The three together compute part t of the public key, of which is decomposed into its higher and lower order components, t_1 and t_0 for the purposes of compression. The public key is a byte encoding of seed ρ and t_1. The private key is a byte encoding of seed ρ, seed K for use in signing, a hash tr of the public key, secret vectors s_1 and s_2, and t_0.

2.3.2 ML-DSA Signing

The signing algorithm ML-DSA.Sign takes a private key, a formatted message, and a context string (a byte string of 255 or fewer bytes) as input and outputs a signature encoded as a byte string.

Algorithm 8: ML-DSA.Sign(sk, M, ctx)

Input : private key $sk \in \mathbb{B}^{32+32+64+32\cdot((\ell+k)\cdot \text{bitlen}(2\eta)+dk)}$
Input : message $M \in \{0,1\}^*$
Input : context string ctx (a byte string of 255 or fewer bytes)
Output: signature $\sigma \in \mathbb{B}^{\lambda/4+\ell\cdot 32\cdot(1+\text{bitlen}(\gamma_1-1))+\omega+k}$

if $|ctx| > 255$ **then**
  return $\perp$ `/* return an error indication if the context string is too long */`
$rnd \leftarrow \mathbb{B}^{32}$ `/* for the optional deterministic variant, substitute` $rnd \leftarrow \{0\}^{32}$ `*/`
if $rnd = \text{NULL}$ **then**
  return $\perp$ `/* return an error indication if random bit generation failed */`
$M' \leftarrow \text{BytesToBits}(\text{IntegerToBytes}(0,1) \,\|\, \text{IntegerToBytes}(|ctx|,1) \,\|\, ctx) \,\|\, M$
$\sigma \leftarrow$ ML-DSA.Sign_internal(sk, M', rnd) **return** σ

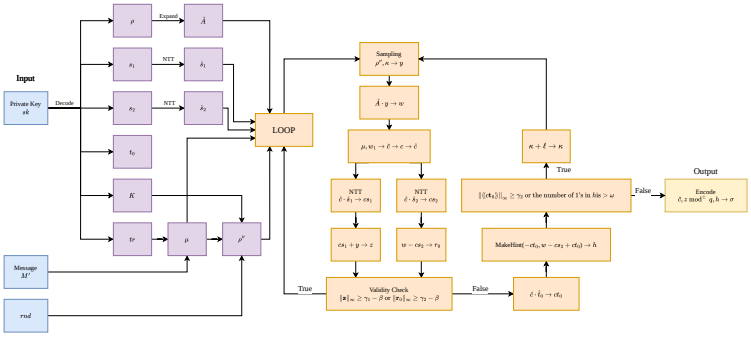


Fig. 7: ML-DSA Internal Signing Algorithm ML-DSA.Sign_internal

From the private key, the signer extracts the following: public random seed ρ, private random seed K, the hash of the public key tr, secret polynomial vectors s_1 and s_2, and lower order component of part t of the public key t_0. Seed ρ is then reexpanded into matrix A as in the key generation algorithm. Message M is concatenated to tr and hashed into 64 byte message representative μ. A 64 byte seed ρ'' is then produced for private randomness during each signing operation by hashing into 64 bytes a concatenation of K, random 32 byte string value rnd, and μ.

Following that, a rejection sampling loop that produces a signature is repeated until a valid one is created, which is then encoded as a byte string and output. The main computations involved in the loop are as follows:

- Generate vector y from using the ExpandMask function with ρ and counter κ as inputs.
- Commitment w_1 is computed by taking the higher order component of w, computed through $A \cdot y$.
- w_1 and μ are concatenated and hashed to produce the commitment hash $\tilde{c}$. $\tilde{c}$ is then used to pseudorandomly sample a polynomial c that has coefficients $\{-1, 0, 1\}$ and Hamming weight τ.
- Response $z = y + cs_1$ is computed. r_0 is also computed by taking the lower order components of $w - cs_2$. Both are used in validity checks that, if failed, restart the loop.
- If the validity checks pass, hint polynomial h is computed using the MakeHint function, which will allow the later verifier to reconstruct w_1 and other components of the signature correctly. After running more checks to see whether the coefficients in ct_0 are within a certain boundary and that the number of 1's in h does not exceed ω, assuming both checks pass, the final signature consisting of a byte encoding of commitment hash $\tilde{c}$, the response z, and the hint h will be output.

2.3.3 ML-DSA Verification

The verification algorithm ML-DSA.Verify takes a public key, a message, a signature, and a context string as input and outputs a boolean value that returns true if the signature is valid with respect to the message and public key and false if it is invalid.

Algorithm 9: ML-DSA.Verify(pk, M, σ, ctx)

Input : public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$
Input : message $M \in \{0,1\}^*$
Input : signature $\sigma \in \mathbb{B}^{\lambda/4+\ell\cdot 32\cdot(1+\text{bitlen}(\gamma_1-1))+\omega+k}$
Input : context string ctx (a byte string of 255 or fewer bytes)
Output: Boolean

if $|ctx| > 255$ **then**
  return $\perp$ `/* return an error indication if the context string is too long */`
$M' \leftarrow$ BytesToBits(IntegerToBytes$(0,1)$ $\|$ IntegerToBytes$(|ctx|,1)$ $\|$ ctx) $\|$ M **return** ML-DSA.Verify_internal(pk, M', σ)

The verifier starts by extracting public random seed ρ and vector t_1 from the public key, then the commitment hash $\tilde{c}$, response z, and hint h from the signature. h is checked to see if it was encoded properly; if not, then the algorithm returns false. Otherwise, matrix A is expanded from ρ, tr is hashed from the public key and concatenated with the message input M', then hashed again to create message representation μ. The verifier's challenge c is computed from $\tilde{c}$ as in the signing algorithm.

The verifier then attempts to reconstruct the signer's commitment w_1'. In the verifier algorithm, w_1' requires w'_{approx}, which is computed through $A{\cdot}z{-}c{\cdot}(t_1{\cdot}2^d)$. w_1 is reconstructed by running the algorithm UseHint, which takes w'_{approx} and recovers the high bits of the w_1 calculated during signing using h. The result is concatenated to μ and hashed to create $\tilde{c}'$.

Finally, the verifier checks that response z is valid (by checking whether all coefficients in z are smaller than bound $\gamma_1 - \beta$) and that $\tilde{c}$ matches with $\tilde{c}'$. If both checks succeed, the verifier returns true. Otherwise, it returns false.

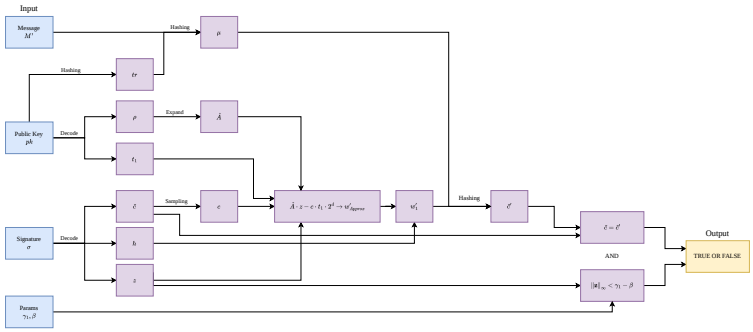


Fig. 8: ML-DSA Internal Verification Algorithm ML-DSA.Verify_internal

2.4 Prior Visualizations

2.5 The Effectiveness of Visualizations

Multiple studies have been conducted on the viability of digital and algorithm visualizations as learning tools or as aids to learning. Among them, results vary depending on the referred visualization tools, subjects of study, measures of effectiveness, metrics for learning, and so on. However, it can be inferred that some tools perform better than others for the purposes of helping their users learn their relevant topics and what characteristics makes those tools work better.

2.5.1 A Meta-Study of Algorithm Visualization Effectiveness

A meta study conducted on the effectiveness of algorithm visualizations in educational scenarios [11] takes into account the objective of each visualization, the individual or groups with that objective, the particular algorithm visualization artifact used, and the particular target algorithm to be visualized, to define the effectiveness of a visualization as 'the extent to which an algorithm visualization artifact assists an individual or group in fulfulling their objective'. They are grouped based on independent variables (factors the study posits to cause effectiveness) and dependent variables (the ways in which a study measures effectiveness). By independent variables, they are grouped into four theories of effectiveness (including some overlap): Epistemic Fidelity, Dual-coding, Individual Differences, and Cognitive Constructivism.

- **Epistemic Fidelity** assumes that humans use symbolic models of the physical world as basis for their reasoning and actions [17]. It also assumes that graphics are effective in encoding an expert's mental model of an algorithm, which leads to effective transferal of the mental model to the viewer. The higher the quality, the 'fidelity' of the model, the more efficient the transfer is meant to be, which leads to more ease on the viewer's part in internalizing the model and presumably, the target knowledge.
- **Dual-coding** follows from the assumption that cognition consists largely of the activity of two partly interconnected but functionally independent systems, one that encodes verbal events and another that encodes non-verbal events. Visualizations that incorporate both should allow viewers to built both representations in the brain, and thus facilitate the transfer of target knowledge better than a visualization that only uses one.
- **Individual Differences** asserts that measurable differences in human ability and learning styles will lead to a measurable performance difference in scenarios of visualization use.
- **Cognitive Constructivism** holds that indivisuals construct their own individual knowledge from subjective experiences. They can actively construct new understandings by actively engaging with their environment or interpreting new experiences through the context of what they already know.

Of the four, Cognitive Constructivism holds the largest group (14 out of 24 studies analysed used it). Additionally, the studies showed the highest percentage (71%) of statistically significant results within studies, followed by Individual Differences and Dual-coding (both 50%, each with only 2 studies each using them), with Epistemic Fidelity showing the least (30% out of 10 studies). A more thorough analysis relating to measuring the effects of effort and levels of engagement [18, 19] shows that Cognitive Constructivism has the most basis, in that participants who actively engaged in activities such as making predictions when viewing a visualization led to a more significult result, as opposed to the equalization of activities leading to non-significant results. In essense, how viewers actually use a visualization matters more than what the visualization actually is.

As for dependent variables, 2 studies measured effectiveness in terms of the participants' success at solving debugging and tracing problems, while the other 22 measured effectiveness by knowledge acquired. Of the latter, the knowledge acquired for each study can be divided into conceptual (an abstract understanding of the properties of an algorithm) and procedural (an understanding of the step-by-step behavior of an algorithm.), as well as some combination of both. An analysis of the results based on type of knowledge acquired implies that procedural knowledge is more sensitive to the effectiveness or lack thereof of visualizations. However, it should be noted that the studies only considered a very limited range of dependent variables, almost exclusively relying on the usage of pre- and post- tests or merely post-tests to come to that conclusion.

3 Latviz

The visualization tool Latviz is a web application that provides visualizations for ML-KEM and ML-DSA, meant to help users more easily understand the two through interactive visualizations.

3.1 Architecture

Latviz is built using Next.js, a fullstack React-based framework, along with Tailwind CSS for the frontend and primarily Typescript for the backend processes. The ML-KEM and ML-DSA algorithms are implemented using a modification of an existing Javascript library 'pqc' that allows for the display of the inner workings of both algorithms as opposed to only the outputs.

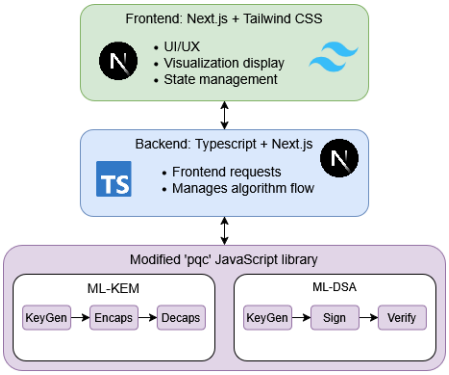


Fig. 9: Latviz fullstack architecture

3.2 Visualization Flow

The visualizations for both algorithms follow the same general flow. After opening the pages for ML-KEM or ML-DSA, the first thing the user sees is a short explanation on the chosen algorithm; what it is, what it is set to replace, and what its difficulty is based on. Below is what the section looks like for ML-DSA (for the sake of brevity, the other sections will also be represented by their ML-DSA counterparts):

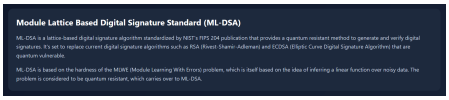


Fig. 10: ML-DSA about component

ML-DSA is described in terms that should be familar to most people with an awareness of cryptography. Rather than focusing on the underlying lattice mathematics, the description emphasizes practical concepts such as signature generation, signature verification, quantum resistant security, and the relationship between ML-DSA and existing algorithms like RSA and ECDSA. This keeps the introduction accessible while preserving the key ideas needed to understand its role in modern cryptographic systems.

After that is a section that briefly explains each stage of the chosen algorithm and provides an animation showing the scheme in use.

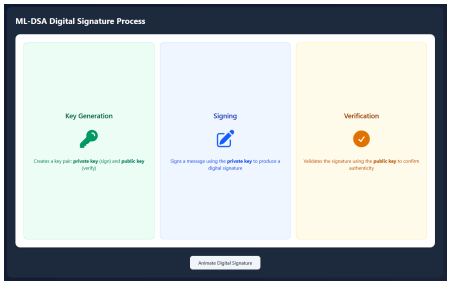


Fig. 11: ML-DSA Process

The section first shows a short description of the purpose of each stage of either algorithm. For ML-DSA, this is key generation, signing, and verification. If the user clicks the button on the bottom of the section, an animation will play that shows the scheme being used in a simplified abstraction of a real world example.

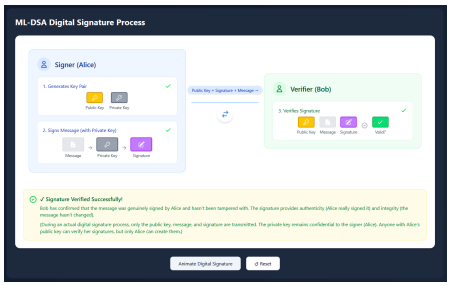


Fig. 12: ML-DSA Process Animation

Below that is a section that displays the an instance of the algorithm in full, including the variables and functions involved in its inner workings. Within it are four different components that each have different functions.

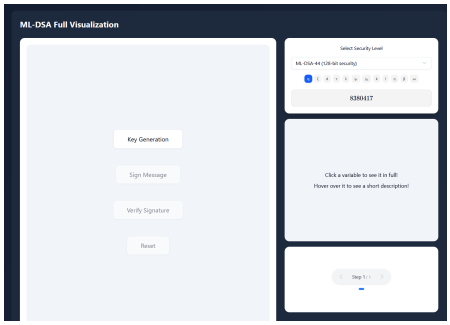


Fig. 13: ML-DSA Full Animation Section

Starting from the top-right is the security level selection component. The user can pick between the security levels ML-DSA-44, ML-DSA-65, and ML-DSA-87, though it defaults to the lowest security level. Beneath the selection, the parameters of the chosen security level are displayed. Whenever a security level is chosen, the visualization is reset and a new instance of ML-DSA is created according to the new security level.

(a) Security level selection (b) Parameters per security level

Fig. 14: Security Level Selection Component

The leftmost component is the main part of this section. It contains four buttons. The first three lead to visualizations of the three stages of ML-DSA. The visualizations come in the form of step-by-step abstractions of the main algorithms of each stage, complete with explanations on the relevant variables and functions.

In ML-DSA specifically, there is an input section for the signing stage that only appears after the key generation stage has been gone through. It needs to be filled in before the signing stage can be accessed, and it is the only actual input from the user that can be used for running an instance of ML-DSA aside from security level selection.

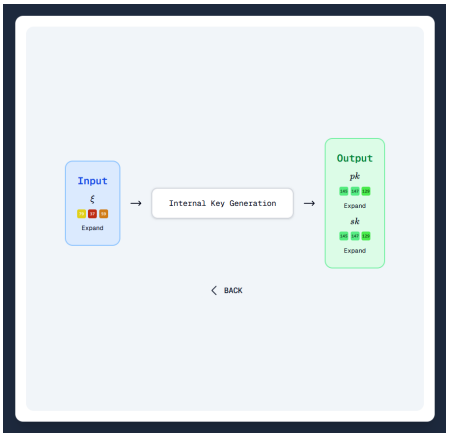
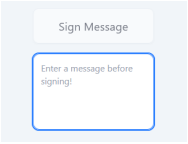
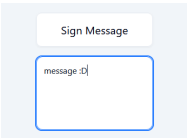


Fig. 15: ML-DSA Full Animation Section



(a) Empty message input

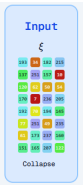


(b) Filled message input

Fig. 16: Input for Sign Message

As interactivity forces engagement and engagement leads to more effective learning [11], these visualizations are interactive. Each variable is displayed in the form of a colored matrix which can be expanded on click to show more values. When hovered on, a popup will appear beside it that describes what it is and provides extra details relating to what it is used for or how it is calculated.

(a) Unexpanded variable



(b) Expanded variable

(c) Popup description

Fig. 17: Variables in the form of colored matrices

Additionally, when a variable is clicked, the component in the middle of the right column will show that variable in scrollable form. This allows for the entire variable to be visible instead of only the first 32 values. There is also an option for the variable to be visible in array form for easier viewing of the values.

(a) Grid view (b) Array view

Fig. 18: Full variables in the side component

When a function is clicked on, it will lead to a visualization of that function. For example, the internal key generation function seen in Figure 14 (more formally known as ML-DSA.KeyGen_internal) will show the following:

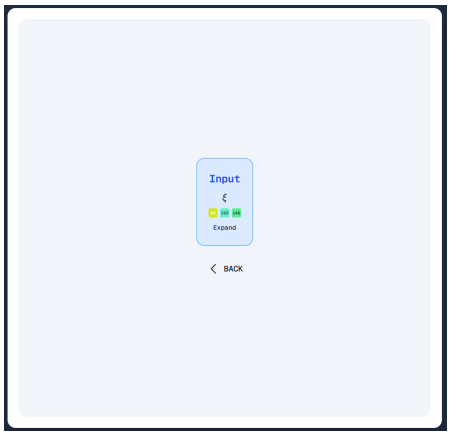


Fig. 19: ML-DSA Full Animation Section (Step 1)

The figure above shows the first step of the function, the input. Subsequent steps can be shown and removed by clicking on the right and left arrows on the lower-right component. Instead of showing the entire function at once, this serves to split the function into more easily followed steps where one part clearly leads to another.

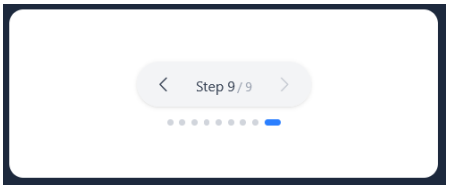


Fig. 20: Stepping Component

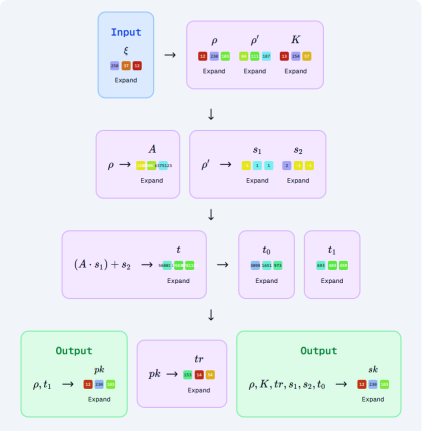


Fig. 21: ML-DSA Full Animation Section (Step 9)

For ML-DSA specifically, there is also a loop function that visualizes each iteration of the valid signature generation loop, elaborating on where and why each specific iteration prior to the last one is invalid.

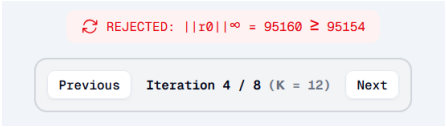


Fig. 22: Sign Loop Iteration Control

4 Evaluation and Results

This section describes the method used to evaluate the effectiveness of Latviz as a visualization tool for ML-KEM and ML-DSA, as well as the results of the evaluation.

4.1 Methods

To evaluate the effectiveness of Latviz as a visualization tool for ML-KEM and ML-DSA, a user survey of Latviz users was conducted. The aim of the survey was to measure if the tool helped users in their understanding of ML-KEM and ML-DSA, if it was effective in doing so, and which parts were found to be adequate or not for the learning process. The group of correspondents consists of 16 people with varying degrees of understanding relating to cryptography, post-quantum cryptography, and lattice-based cryptography prior to using Latviz.

The survey consists primarily of 5-point Likert-scale questions ranging from 'Strongly Agree' to 'Strongly Disagree' and focuses on evaluating the following aspects of the visualization: overall understanding of the algorithms, clarity and ease of understanding of the explanations, the effectiveness of the step-by-step visualization approach, comprehension of variables and internal algorithmic processes, the usability and readability of the interface, and the perceived effectiveness of Latviz as a learning tool. In addition, an optional section was added to gather user suggestions or feedback that was not covered by the available questions.

4.2 Results

From

5 Conclusion

References

1. Benioff, Paul. (1980). The computer as a physical system: A microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines. *Journal of Statistical Physics*, 22(5), 563–591. https://doi.org/10.1007/bf01011339
2. Benioff, Paul. (1982). Quantum mechanical hamiltonian models of turing machines. *Journal of Statistical Physics*, 29(3), 515–546. https://doi.org/10.1007/BF01342185
3. Feynman, Richard. (1982). Simulating Physics with Computers. *International Journal of Theoretical Physics*, 21(6/7), 467–488. https://doi.org/10.1007/BF02650179
4. Bennett, C. H. & Brassard, G. (2014). Quantum cryptography: Public key distribution and coin tossing. *Theoretical Computer Science. Theoretical Aspects of Quantum Cryptography – celebrating 30 years of BB84*, 560(1), 7–11. https://doi.org/10.1016/j.tcs.2014.05.025
5. Brassard, G. (2005). Brief history of quantum cryptography: A personal perspective. https://doi.org/10.1109/ITWTPI.2005.1543949
6. Shor, P.W. (1994). Algorithms for quantum computation: discrete logarithms and factoring. *Proceeding 35th Annual Symposium on Foundations of Computer Science*. https://doi.org/10.1109/SFCS.1994.365700
7. National Institute of Standards and Technology. (2024). Module-Lattice-Based Key-Encapsulation Mechanism Standard. *Federal Information Processing Standards Publication*. https://doi.org/10.6028/NIST.FIPS.203

8. National Institute of Standards and Technology. (2024). Module-Lattice-Based Digital Signature Standard. *Federal Information Processing Standards Publication*. https://doi.org/10.6028/NIST.FIPS.204
9. Kalava et al. (2025). Post-Quantum Security for Web Applications. https://pqc.nithinram.com/Post-Quantum
10. Kang, D. (2026). ML-KEM from Scratch. *Github.io*. https://hulryung.github.io/ml-kem-notebooks/intro.html
11. Hundhausen, C. D., Douglas, S. A., & Stasko, J. T. (2002). A Meta-Study of Algorithm Visualization Effectiveness. *Journal of Visual Languages & Computing*, 13(3), 259–290. https://doi.org/10.1006/jvlc.2002.0237
12. Avanzi et al. (2021). CRYSTALS-Kyber. https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf
13. Bai et al. (2021). CRYSTALS-Dilithium. https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf
14. Barker, E. & Kesley, J. (2015). Recommendation for Random Number Generation Using Deterministic Random Bit Generators. *NIST Special Publication 800-90A*. https://doi.org/10.6028/nist.sp.800-90ar1
15. Barker et al. (2018). Recommendation for the Entropy Sources Used for Random Bit Generation. *NIST Special Publication 800-90B*. https://doi.org/10.6028/NIST.SP.800-90B
16. Barker et al. (2025). Recommendation for Random Bit Generator (RBG) Constructions. *NIST Special Publication 800-90C*. https://doi.org/10.6028/nist.sp.800-90c
17. Allen, N. & Herbert, A, S. (1972). *Human Problem Solving*. Prentice-Hall, Englewood Cliffs, NJ.
18. Byrne, M. D., Catrambone, R., & Stasko, J. T. (1999). Evaluating animations as student aids in learning computer algorithms. *Computers & Education*, 33(4), 253–278. https://doi.org/10.1016/s0360-1315(99)00023-8
19. Judith, S. G., (1996) Pedagogic Aspects of Algorithm Animation. Unpublished Ph.D. dissertation, Computer Science, University of Colorado
20. Stasko, J., Badre, A., & Lewis, C. (1993). Do Algorithm Animations Assist Learning? An Empirical Study and Analysis. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems - CHI '93*, 61–66. https://doi.org/10.1145/169059.169078
21. Langlois, A. & Stehle, D. (2012). Worst-Case to Average-Case Reductions for Module Lattices. https://eprint.iacr.org/2012/090